

Phase 2 Training Task

Real Database, API Security, and Backend Workflow APIs

API-only phase: React frontend will be developed in Phase 3

Item	Details
Audience	Trainees continuing from Phase 1: TraineeManagement.Api
Duration	5 working days
Backend Stack	C#, ASP.NET Core Web API, Entity Framework Core, MySQL
Security Scope	JWT authentication, password hashing, protected APIs, CORS, OWASP API security basics
Important Note	Do not create a new backend project. Extend the existing Phase 1 project.

1. Training Objective

In Phase 2, trainees will extend the same Trainee Management API built in Phase 1. The objective is to move from EF Core InMemory DB to a real MySQL database using the EF Core Code First approach, add authentication and API security, and extend the backend system with workflow APIs used in trainee training management.

- Use MySQL as the real database provider.
- Create and apply EF Core Code First migrations.
- Add User/Auth module with password hashing and JWT token generation.
- Protect backend APIs using JWT bearer authentication.
- Add pagination, filtering, structured logging, CORS, and global exception handling.
- Extend the system with Mentor, Learning Task, Task Assignment, Submission, and Review APIs.

Official references: [ASP.NET Core Web API](#) | [ASP.NET Core fundamentals](#)

2. Existing Project to Extend

Continue from the Phase 1 project:

TraineeManagement.Api

Existing Phase 1 APIs:

GET /api/health
GET /api/trainees
GET /api/trainees/{id}
POST /api/trainees
PUT /api/trainees/{id}
DELETE /api/trainees/{id}
GET /api/trainees?search=amit

3. Phase 2 System Extension

Module	Purpose
User/Auth	Login, password hashing, JWT token generation
Mentor	Manage mentors

Learning Task	Create training tasks
Task Assignment	Assign tasks to trainees
Submission	Trainee submits work
Review	Mentor reviews submitted work

4. Recommended 5-Day Plan

Day	Focus
Day 1	Replace EF Core InMemory DB with MySQL using EF Core Code First
Day 2	User/Auth module, password hashing, JWT
Day 3	Protected APIs, CORS, pagination, logging
Day 4	Mentor and Learning Task APIs
Day 5	Task Assignment, Submission, Review APIs, global exception handling, security review

Day 1: MySQL Database with EF Core Code First

Objective: Replace EF Core InMemory DB with MySQL using the EF Core Code First approach.

Task 2.1: Configure MySQL Provider

- Replace UseInMemoryDatabase with MySQL database configuration.
- Read the database connection string from appsettings.json.
- Do not hardcode connection strings inside controllers or services.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
      "server=localhost;port=3306;database=trainee_management_db;user=root;password=your_password;"
  }
}
```

Official references: [EF Core database providers](#) | [MySQL Connector/NET EF Core support](#) | [ASP.NET Core configuration](#)

Task 2.2: Create EF Core Migration

- Create migration for the existing Trainee entity.
- Apply the migration to create the MySQL database schema.  Verify that the Trainees table is created with the expected columns.

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

Official references: [EF Core migrations overview](#) | [Applying EF Core migrations](#)

Task 2.3: Verify Existing APIs with MySQL

- All existing trainee CRUD APIs should work with MySQL.
- Create a trainee, restart the API, and verify the record still exists.  Update README with MySQL setup and migration commands.

```
GET /api/trainees
```

GET /api/trainees/{id}
POST /api/trainees
PUT /api/trainees/{id}
DELETE /api/trainees/{id}

Official references: [Create controller-based web API](#) | [Model validation](#)

Day 1 Deliverables

- MySQL database configured.
- EF Core migration created and applied.
- Existing trainee APIs working with MySQL.
- README updated with database setup steps.
- Swagger/Postman evidence captured.

Day 2: User/Auth, Password Hashing, and JWT

Objective: Add secure login using hashed passwords and JWT-based authentication.

Task 2.4: Create User Entity

Field	Description
Id	Primary key
Username	Unique username
Email	User email
PasswordHash	Hashed password only
Role	Admin, Mentor, Trainee
CreatedDate	Auto-generated
UpdatedDate	Auto-generated

- Create Users table through EF Core migration.
- Do not return PasswordHash in any API response.
- Seed or create at least one Admin user for testing.

Official references: [EF Core migrations overview](#) | [ASP.NET Core authentication overview](#)

Task 2.5: Implement Password Hashing

- Passwords must never be stored as plain text.
- Store only PasswordHash in the Users table.
- Verify the password during login using password verification logic.
- Do not log passwords or password hashes.

Official references: [Hash passwords in ASP.NET Core](#) | [OWASP Password Storage Cheat Sheet](#)

Task 2.6: Create Login API

POST /api/auth/login

Request:

```
{
  "username": "admin",
  "password": "Admin@123"
}
```

Response:

```
{
  "token": "jwt-token-value",
  "expiresIn": 3600,
  "user": {
    "id": 1,
    "username": "admin",
    "role": "Admin"
  }
}
```

Official references: [Configure JWT bearer authentication](#) / [Authentication overview](#)

Task 2.7: Add JWT Configuration

- JWT settings should come from configuration.
- The signing key should not be hardcoded inside controller code. ☑ Token should include useful claims: UserId, Username, Role.

```
{
  "Jwt": {
    "Issuer": "TraineeManagementApi",
    "Audience": "TraineeManagementClient",
    "ExpiryMinutes": 60
  }
}
```

Official references: [JWT bearer authentication](#) / [ASP.NET Core configuration](#)

Day 2 Deliverables

- Users table created through migration.
- Password hashing implemented.
- Login API working.
- JWT token generated successfully.
- Auth-related DTOs created.
- PasswordHash is not returned in responses.

Day 3: Protected APIs, Pagination, CORS, and Logging

Objective: Secure APIs and make list APIs scalable and frontend-ready for Phase 3.

Task 2.8: Protect APIs with JWT

Public APIs	Protected APIs
GET /api/health	GET /api/trainees
POST /api/auth/login	GET /api/trainees/{id}

	POST /api/trainees
	PUT /api/trainees/{id}
	DELETE /api/trainees/{id}

- No token should return 401 Unauthorized.
- Invalid token should return 401 Unauthorized.
- Valid token should allow the API call.
- Clients should send Authorization: Bearer <token>.

Official references: [JWT bearer authentication](#) / [Simple authorization in ASP.NET Core](#)

Task 2.9: Add Pagination, Search, and Status Filter

GET /api/trainees?pageNumber=1&pageSize=10&search=amit&status=Active

```
{
  "pageNumber": 1,
  "pageSize": 10,
  "totalRecords": 25,
  "data": []
}
```

- Use pageNumber and pageSize query parameters.
- Support search and status filter together with pagination.
- Do not return large datasets blindly.  Use Skip() and Take() appropriately.

Official references: [Create web APIs with ASP.NET Core](#) / [EF Core querying data](#)

Task 2.10: Configure CORS for Phase 3 React Frontend

- React frontend will be developed in Phase 3, but backend CORS should be prepared now.
- Allow local React development origins such as http://localhost:3000 and http://localhost:5173.  Do not use unrestricted CORS for production-style code.

Official references: [Enable CORS in ASP.NET Core](#)

Task 2.11: Add Structured Logging

- Log user login success/failure.
- Log trainee created, updated, and deleted events.
- Log record-not-found cases and unexpected exceptions.
- Do not log passwords, JWT tokens, or sensitive data.

Official references: [Logging in ASP.NET Core](#) / [OWASP Logging Cheat Sheet](#)

Day 3 Deliverables

- Trainee APIs protected using JWT.

- Swagger/Postman testing with JWT completed.
- Pagination, search, and status filter working together.
- CORS configured for future React frontend.
- Structured logging added.

Day 4: Mentor and Learning Task APIs

Objective: Extend the backend system with additional modules while following the same clean API structure used for Trainee APIs.

Task 2.12: Create Mentor Module

Field	Description
Id	Primary key
FirstName	Mentor first name
LastName	Mentor last name
Email	Mentor email
Expertise	Technical expertise
Status	Active / Inactive
CreatedDate	Auto-generated
UpdatedDate	Auto-generated

GET /api/mentors
 GET /api/mentors/{id}
 POST /api/mentors
 PUT /api/mentors/{id}
 DELETE /api/mentors/{id}

- Create DTOs, validation, service, and controller.
- Protect APIs using JWT.
- Add migration for Mentor table.

Official references: [ASP.NET Core Web API](#) | [Model validation](#) | [Dependency injection](#)

Task 2.13: Create Learning Task Module

Field	Description
Id	Primary key
Title	Task title
Description	Task description
ExpectedTechStack	Expected technology stack
DueDate	Due date
Status	Draft / Published / Closed
CreatedDate	Auto-generated
UpdatedDate	Auto-generated

GET /api/learning-tasks
 GET /api/learning-tasks/{id}
 POST /api/learning-tasks
 PUT /api/learning-tasks/{id}
 DELETE /api/learning-tasks/{id}

Day 4 Deliverables

- Mentor CRUD APIs completed.
- Learning Task CRUD APIs completed.
- DTOs and validation added.
- APIs protected using JWT.
- MySQL migrations updated.
- Swagger/Postman testing completed.

Day 5: Task Assignment, Submission, Review, Exception Handling, Security Review

Objective: Complete backend workflow APIs and add API production-readiness improvements.

Task 2.14: Create Task Assignment Module

Field	Description
Id	Primary key
TraineeId	Linked trainee
MentorId	Linked mentor
LearningTaskId	Linked learning task
AssignedDate	Assignment date
DueDate	Due date
Status	Assigned / InProgress / Submitted / Reviewed / Completed
Remarks	Optional remarks

POST /api/task-assignments

GET /api/task-assignments

GET /api/task-assignments/{id} PUT

/api/task-assignments/{id}/status

- TraineeId must exist.  MentorId must exist.
- LearningTaskId must exist.
- DueDate should not be before AssignedDate.

Official references: [EF Core relationships](#) | [EF Core querying data](#)

Task 2.15: Create Submission Module

Field	Description
Id	Primary key
TaskAssignmentId	Linked assignment
SubmissionUrl	GitHub/Drive/submission link
Notes	Trainee notes
SubmittedDate	Submission date
Status	Submitted / Resubmitted

POST /api/submissions

GET /api/submissions
GET /api/submissions/{id}

Official references: [ASP.NET Core Web API](#) / [Model validation](#)

Task 2.16: Create Review Module

Field	Description
Id	Primary key
SubmissionId	Linked submission
MentorId	Reviewing mentor
Feedback	Mentor feedback
Score	Optional score
ReviewStatus	Accepted / ChangesRequired / Rejected
ReviewedDate	Review date

POST /api/reviews
GET /api/reviews
GET /api/reviews/{id}

Official references: [EF Core relationships](#) / [Authorization in ASP.NET Core](#)

Task 2.17: Add Global Exception Handling Middleware

- Create centralized exception handling for unexpected errors.
- Return safe, consistent error messages to clients.
- Do not expose stack traces or internal exception details in API responses.

```
{  
  "message": "An unexpected error occurred. Please try again later."  
}
```

Official references: [Handle errors in ASP.NET Core](#)

Task 2.18: OWASP API Security Basics Review

Security Area	Expected Practice
Authentication	JWT validation enabled
Authorization	Protected APIs require token
Password storage	Passwords stored as hash only
Excessive data exposure	DTOs used; password hash not returned
Injection	EF Core used; no unsafe raw SQL
Security misconfiguration	CORS restricted to expected origin
Sensitive data exposure	Secrets not hardcoded in controllers
Error handling	Stack traces not returned
Logging	Passwords and tokens not logged

Official references: [OWASP API Security Top 10 - 2023](#) / [OWASP Cheat Sheet Series](#)

Day 5 Deliverables

- Task Assignment APIs completed.
- Submission APIs completed.
- Review APIs completed.
- Relationships validated.
- Global exception handling added.
- OWASP API security checklist completed. [🔗](#) Final backend demo ready.

5. Final Phase 2 API Scope

GET /api/health

POST /api/auth/login

GET /api/trainees?pageNumber=1&pageSize=10&search=amit&status=Active

GET /api/trainees/{id}

POST /api/trainees

PUT /api/trainees/{id}

DELETE /api/trainees/{id}

GET /api/mentors

GET /api/mentors/{id}

POST /api/mentors

PUT /api/mentors/{id}

DELETE /api/mentors/{id}

GET /api/learning-tasks

GET /api/learning-tasks/{id}

POST /api/learning-tasks

PUT /api/learning-tasks/{id}

DELETE /api/learning-tasks/{id}

POST /api/task-assignments

GET /api/task-assignments

GET /api/task-assignments/{id}

PUT /api/task-assignments/{id}/status

POST /api/submissions

GET /api/submissions

GET /api/submissions/{id}

POST /api/reviews

GET /api/reviews

GET /api/reviews/{id}

6. Final Submission Checklist

Item	Expected
Backend source code	Updated TraineeManagement.Api
MySQL database setup	Documented in README
EF Core migrations	Included in repository
JWT login	Working
Protected APIs	Require valid token

Password hashing	Implemented
Pagination	Implemented
CORS	Configured for Phase 3 frontend
Logging	Implemented
Global exception handling	Implemented
Postman collection / Swagger evidence	Included
Security checklist	Completed
README	Updated with setup and usage steps

7. README Must Include

- Project name and technology stack.
- Backend setup steps.
- MySQL setup steps.
- EF Core migration commands.
- Login credentials for testing.
- JWT usage instructions.
- API list and sample JSON requests/responses.
- Known limitations.
- Security checklist.
- Next improvement areas.